ToyDB: index construction summary (1)

Part 3, Part A: Methods 1 and 2 | 2 November 2026

Part 3 builds a B+ tree index on the Student file, keyed on roll-no, using the supplied AM layer. Three construction methods are to be compared. This document covers what is finished so far: **Method 1** (index an existing file), **Method 2** (build the file and the index together), the shared benchmark and validation code, the test results and the preliminary measurements. **Method 3** (bulk loading from a pre-sorted file) is Part B and has not been started.

> **Status.** Part A complete: both methods build valid indexes that the normal AM search APIs can read. All 4 test runs pass, and the core tests also pass under AddressSanitizer and UBSan. Preliminary results are below. Part B (Method 3) is pending and plugs into the existing benchmark without changing it.

1. What was built

File (toydb/amindex/)	Purpose
`ixbuild.c`	Method 1, Method 2, and the method table `IX_Methods[]` where Method 3 is registered
`ixindex.h/.c`	Shared code: roll-no key extraction, measurement (time + PF counters), query sets and runner, tree inspection, validation
`amwrap.h`	Includes the K&R `am.h` safely from modern C and adds AM prototypes
`ixbench.c`	Benchmark CLI: builds with method K, checks the tree, validates, times the query file; one JSON line per run
`testindex.c`	Correctness tests (real data and a synthetic 150,000-record file)
`run_tests.py`, `run_experiments.py`, `experiments.json`	Test runner and configurable experiment runner; results go to `outputs/index-construction/`
`README.md`	Build/run instructions, index file format and the Part B handover

Reused without changes: the PF layer (part 1) for paging and I/O counters, and the slotted-page Student file (part 2) for records and RIDs. The only existing file edited is `toydb/amlayer/amscan.c` (three bug fixes, section 5).

2. The two methods

Method	Steps	What "construction" measures
1. Existing file	Open the existing Student file; `AM_CreateIndex`; sequential scan (`SP_ScanNext`); for each record extract roll-no and RID and call `AM_InsertEntry`; close both files.	Create + scan + inserts + closing (flushes dirty pages). Writing the file first and opening it are reported as setup.
2. Incremental	Create an empty Student file and an empty index; for each dataset record call `SP_InsertRec` (gives the RID), then `AM_InsertEntry`; close both files.	Everything: the file and the index are built together, so there is no separate setup.

3. Key design decisions

- **Roll-no** is field 2 of each line (e.g. `95302001`, `96D07001`). All 17,813 values are 8 characters and some contain letters, so the key is an AM `'c'` key of length 8, compared in byte order.
- **RID** is part 2's `page × 65536 + slot`, a non-negative `int`, which is exactly AM's `recId` type.
- **Duplicates:** the data has 3 roll-nos that appear twice. AM stores one key with a list of RIDs, and an EQUAL search returns all of them (17,813 entries, 17,810 distinct keys).
- **Records without a roll-no** are stored but not indexed and are counted as skipped (none in the real data).
- **Index file** is the unmodified AM format: PF file `<studentFile>.0`, opened with `PF_OpenFile`.

Full options and reasoning: `outputs/design-decisions/part3-design-decisions.md`.

Preliminary results

Dataset `data/student.txt` (17,813 records). 2,500 point queries: 2,000 existing roll-nos and 500 absent ones, generated once with seed 42 and run in the same order for every method. LRU replacement. Each value is the median of 5 independent runs; query time is the median of 5 repetitions per run, each starting with none of the index's pages in the buffer pool.

Pages accessed = PF logical reads (page requests to the buffer pool). **Disk reads/writes** = PF physical I/O. Construction and query numbers come from separate counter snapshots.

Required comparison table (20-frame buffer pool)

Metric	Method 1 (existing)	Method 2 (incremental)
Records indexed	17,813	17,813
Construction time	**9.56 ms**	**17.18 ms**
Setup outside construction	8.01 ms	0.00 ms
Setup + construction (end to end)	17.58 ms	17.18 ms
Construction pages accessed	35,931	52,838
Construction disk reads / writes	519 / 176	77 / 640
Query completion time (2,500 queries)	**3.23 ms**	**3.09 ms**
Query pages accessed	**9,522**	**9,522**
Pages accessed per query	3.81	3.81
Query disk reads	2,060	2,060
Tree height / internal / leaf pages	2 / 1 / 109	2 / 1 / 109
Mean leaf fill	64.4%	64.4%

Effect of buffer pool size

Buffer frames	M1 construction	M2 construction	M1 constr. disk R / W	M2 constr. disk R / W	Query time M1 / M2	Query disk reads
20	9.56 ms	17.18 ms	519 / 176	77 / 640	3.23 / 3.09 ms	2,060
50	10.39 ms	15.59 ms	482 / 139	39 / 602	2.44 / 2.45 ms	1,375
200	9.41 ms	16.32 ms	463 / 120	15 / 578	1.25 / 1.29 ms	110

Pages accessed do not depend on the pool size (35,931 / 52,838 for construction, 9,522 for queries). With 200 frames the whole 110-page index fits in the pool, so each query page is read from disk only once.

What the numbers show

- **Methods 1 and 2 build the same tree.** Method 1's sequential scan returns records in RID order, which is the order Method 2 inserted them, so AM receives the same key sequence. Query cost is therefore identical, and the methods differ only in construction cost.
- **Construction alone:** Method 1 is about 45% faster and makes about 32% fewer page requests, because the Student file already exists. **End to end** (including writing the file) the two cost about the same.
- **I/O pattern:** Method 1 re-reads the whole Student file (about 464 pages) during its scan. Method 2 reads almost nothing from disk but writes more, because Student-file and index pages compete for the same buffer pool.
- **Per query:** about 4 page requests for a hit (root lookup for the leftmost leaf, root, leaf, leaf again in `AM_FindNextEntry`) and about 3 for a miss.
- Leaves are only 64% full because inserts split leaves in half. A sorted bulk load (Method 3) should give fuller leaves and fewer pages; that is the main comparison for Part B.

Correctness and AM fixes

4. Tests (all passing)

Check	How
Existing, non-existing and several roll-nos	Explicit searches after reopening the index, including the 3 duplicated roll-nos
Every RID points to the right Student record	`IX_Validate`: for **every** distinct key, the EQUAL scan returns exactly as many distinct RIDs as the dataset has records with that roll-no, and each RID's record has that roll-no; plus 202 absent keys below, between and above the real ones
Entry count = records indexed	Full AM scan (17,813 entries in key order) and a walk of the whole tree
Close and reopen	Validation and queries reopen the index; queries are repeated on a second reopen and must give identical answers and page counts
B+ tree invariants	`IX_InspectTree`: page types, sorted keys, separator ranges, equal leaf depth, complete leaf chain, leftmost leaf = page 2
Index valid during Method 2	Invariants and searches checked at 20 points while records are still being inserted
Both methods agree	Identical answers (RID checksum) for 600 queries
Node splitting and multiple levels	Synthetic 150,000-record file (random order, duplicates, records without a roll-no) gives a 3-level tree: 5 internal pages, 894 leaves
No page left fixed	Tests run with 3, 20 and 200 buffer frames; 3 is AM's minimum (a root split fixes 3 pages)

Log: `outputs/index-construction/test-results.txt`. Run with `python3 toydb/amindex/run_tests.py`.

5. Bugs found and fixed in the supplied AM layer

Bug	Effect	Fix (in `amscan.c`)
`AM_OpenIndexScan` pushes the search path onto a 50-entry stack that only insert/delete empty	Once the tree has 2+ levels, the 51st search writes past the end of `AM_Stack` (found with AddressSanitizer)	FIX 1: empty the stack after the search
Absent key that sorts after the last key of a leaf: the next leaf's header overwrites the current one before its page number is used	The wrong page is unfixed, so the search fails with `AME_PF` and the next leaf stays fixed in the buffer	FIX 2: save the next leaf's page number first
Same pattern in `AM_FindNextEntry` for `NOT_EQUAL` scans	Same as above	FIX 3

Each fix is a few lines and is marked `PART 3 FIX n`; search results are otherwise unchanged. Separately, `toydb/amlayer/pf.h` still says pages are 1020 bytes while the PF layer uses 4096. That file was not edited: the amindex Makefile compiles copies of the AM sources so they use the correct header, and a test checks that the root's `maxKeys` is 340 (the 4096-byte value).

6. How to build and run

```
make -C toydb/amindex                     # builds ixbench and testindex
python3 toydb/amindex/run_tests.py        # all tests  -> outputs/index-construction/test-results.txt
python3 toydb/amindex/run_experiments.py  # benchmark  -> outputs/index-construction/comparison.md

# one method by hand
./ixbench queries --data ../../data/student.txt --hits 2000 --misses 500 --seed 42 --out work/q.txt
./ixbench run --method 1 --data ../../data/student.txt --queries work/q.txt --workdir work --validate
```

Build under Linux/WSL. Generated Student and index files go to a temporary or `--workdir` directory and are deleted after each run; `data/student.txt` is only read.

Handover for Part B and remaining work

7. Adding Method 3 (bulk loading)

1. Write `ixbulk.c` with `int IX_Method3(const IX_BuildSpec *spec, IX_BuildResult *r)`: sort a copy of the dataset by key, write the sorted Student file (measured as setup), bulk-build the index from the sorted (key, RID) pairs (measured as construction, including closing the files), and set the record counts.
2. Register it in `IX_Methods[]` (`ixbuild.c`) and add `build/ixbulk.o` to the Makefile.
3. Check it with `ixbench run --method 3 --validate`, add `3` to `"methods"` in `experiments.json` and rerun `run_experiments.py`. The table gains a Method 3 column with the same queries and the same procedure.

8. Index format rules Method 3 must keep

Rule	Reason
Root is always page 0	AM searches start at the file's first page, and inserts detect a root split by page number 0
Leftmost leaf is page 2 when the root is internal	Hard-coded in `GetLeftPageNum` for full and `<`, `<=`, `!=` scans; checked by `IX_InspectTree`
Leaf: 24-byte header, then sorted `[8-byte key][2-byte list offset]`; RID nodes `[4-byte RID][2-byte next]` grow down from the page end	At most 254 distinct keys per leaf
Internal: 8-byte header, then `[child0][key1][child1]...`, `maxKeys = 340`	Child i holds keys from key_i (inclusive) up to key_{i+1}; later AM inserts must still split correctly
All RIDs of one key stay in one leaf	AM's EQUAL scan stops at the end of the leaf

Method 3's Student file is sorted, so its RIDs (and RID checksum) will differ from Methods 1 and 2; correctness is shown by `--validate` and by equal found/RID counts. Full details: `toydb/amindex/README.md`.

9. Remaining work and limitations

- **Pending:** Method 3 (bulk loading), its comparison against Methods 1 and 2, and the final Part 3 summary.
- Only point queries are benchmarked; range queries are not measured.
- AM allows about 670 RIDs per key (the real data has at most 2).
- Times are in milliseconds and vary between runs, so medians are reported; page counts are deterministic and are the more reliable metric.
- The supplied AM test programs (`amlayer/test*.c`) have not been rebuilt.

10. Files

Location	Content
`toydb/amindex/`	All Part A code, tests and runners
`toydb/amlayer/amscan.c`	Three marked bug fixes
`outputs/index-construction/`	`comparison.md`, `summary.csv`, `results.csv`, `raw_results.jsonl`, `queries.txt`, `config.json`, test logs, this summary's source
`outputs/design-decisions/part3-design-decisions.md`	Design decisions with options and evidence